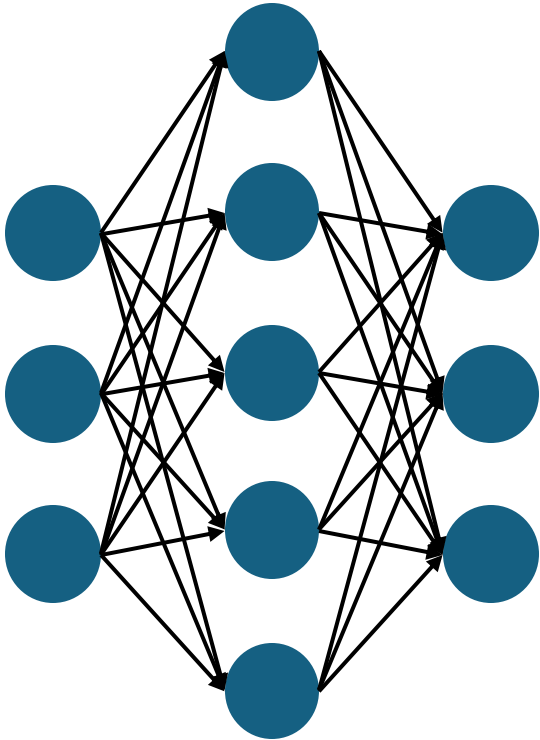


Consider LoRA a Kind of Fast Weight and Temporary Memory in Familiarity Effects

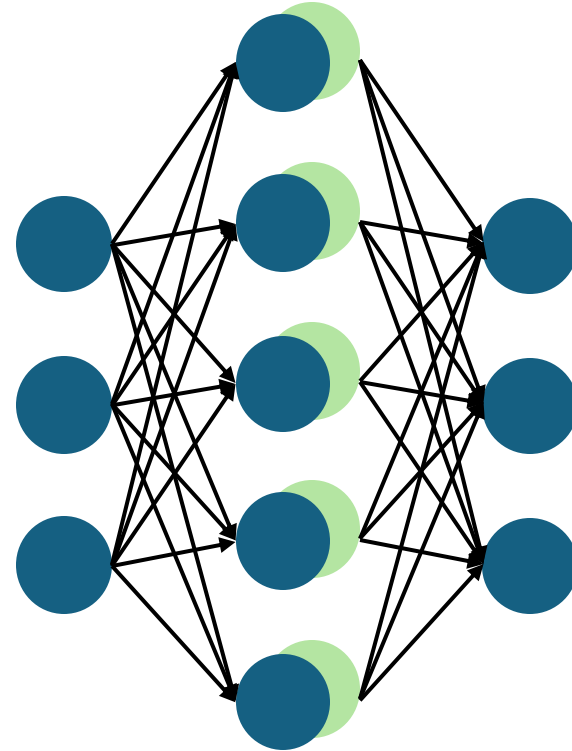
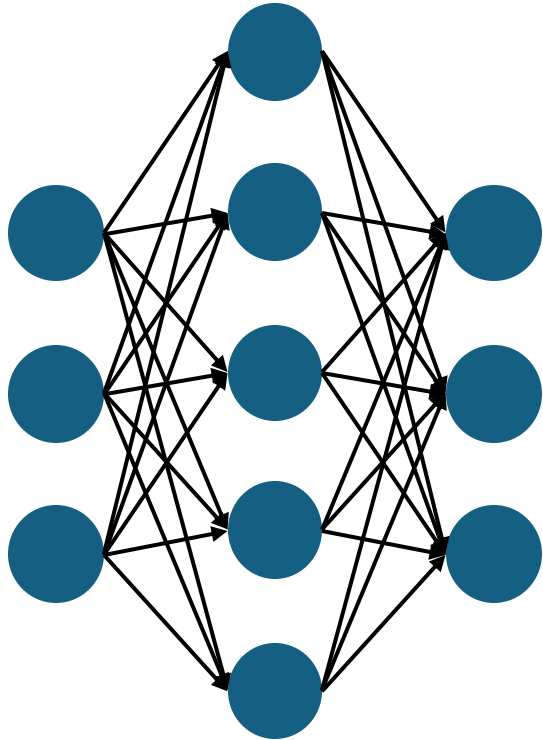
Fast Weight



Reference:

Hinton, G.E. and Plaut, D.C., 1987, July. Using fast weights to deblur old memories. In *Proceedings of the ninth annual conference of the Cognitive Science Society* (pp. 177-186).

Fast Weight



Reference:

Hinton, G.E. and Plaut, D.C., 1987, July. Using fast weights to deblur old memories. In *Proceedings of the ninth annual conference of the Cognitive Science Society* (pp. 177-186).

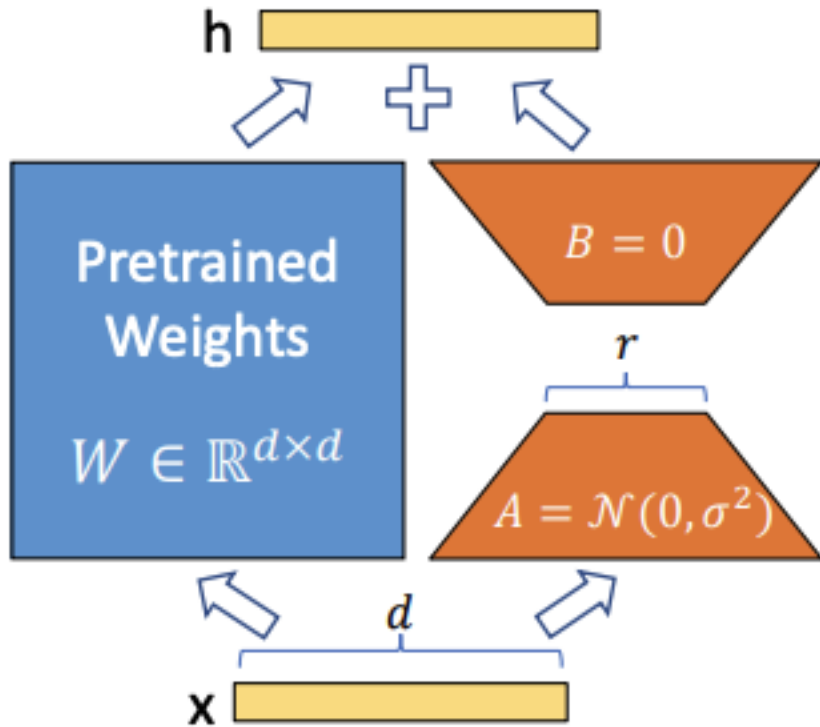
Fast Weight

- A weight that has a larger learning rate than other weights (slow weight) of a neural network
- Can be an additional component of a neural network
- Slow weights hold the long-term knowledge of the network, while fast weights store temporary memories and the most recent information

Reference:

Hinton, G.E. and Plaut, D.C., 1987, July. Using fast weights to deblur old memories. In *Proceedings of the ninth annual conference of the Cognitive Science Society* (pp. 177-186).

LoRA (Low-Rank Adaptation)



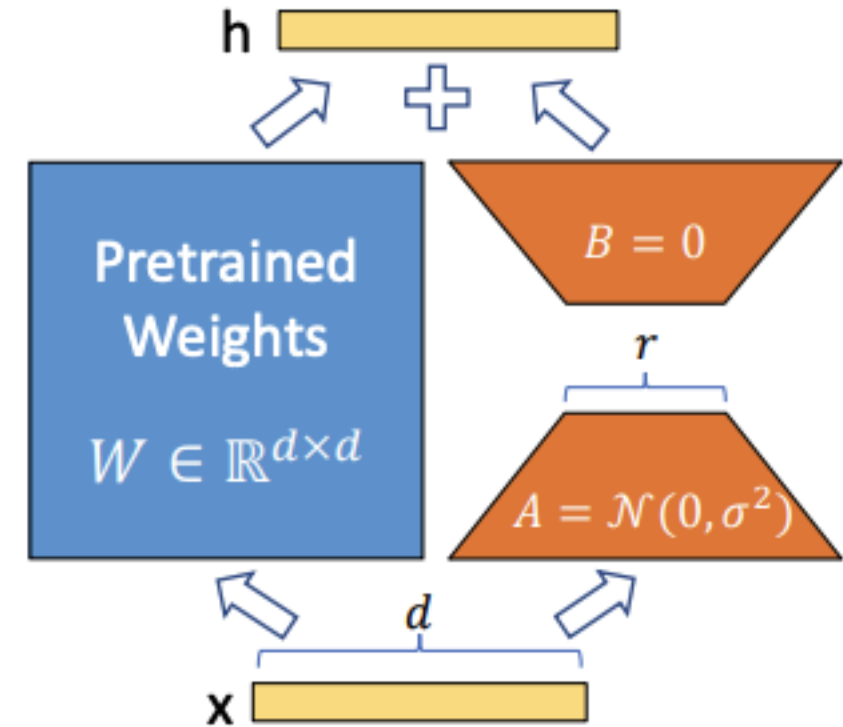
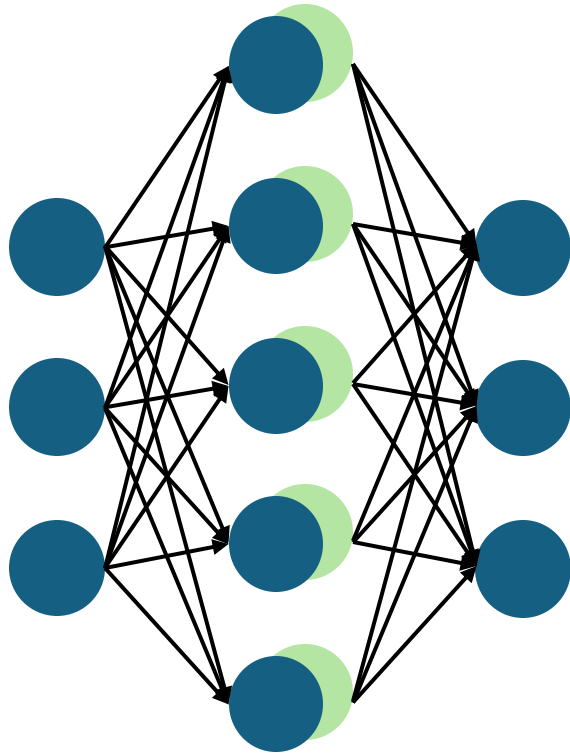
- During fine-tuning, LoRA freezes the initial weights (W) and just learns an additive modification to the initial weights (BA)
- Rank of BA (r) is much smaller than the input dimension (d). $r \ll d$.
- After fine-tuning, the new weights can be updated: $W_{\text{new}} = W + BA$

Reference:

Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. "Lora: Low-rank adaptation of large language models." *arXiv preprint arXiv:2106.09685* (2021).

Connection between LoRA and Fast Weight

- During fine-tuning, the learning rate of the pre-trained weights is 0. Considering the initial weights as slow weights and LoRA weights as fast weights.



Reference:

Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. "Lora: Low-rank adaptation of large language models." *arXiv preprint arXiv:2106.09685* (2021).

Familiarity Effects

- Familiarity effects:

Refer to cognitive or perceptual phenomena where exposure to a stimulus is enhanced

- Familiarity training in the context of computer vision:

Take training a model with the same image as a kind of familiarity training

- Relation with Fast Weight and LoRA:

Assign each image its own specific LoRA weights to store the temporary memory

- Expectation of familiarity training:

After training, the model can perform (1) noise robustness, (2) faster recognition, (3) neurophysiological characteristics

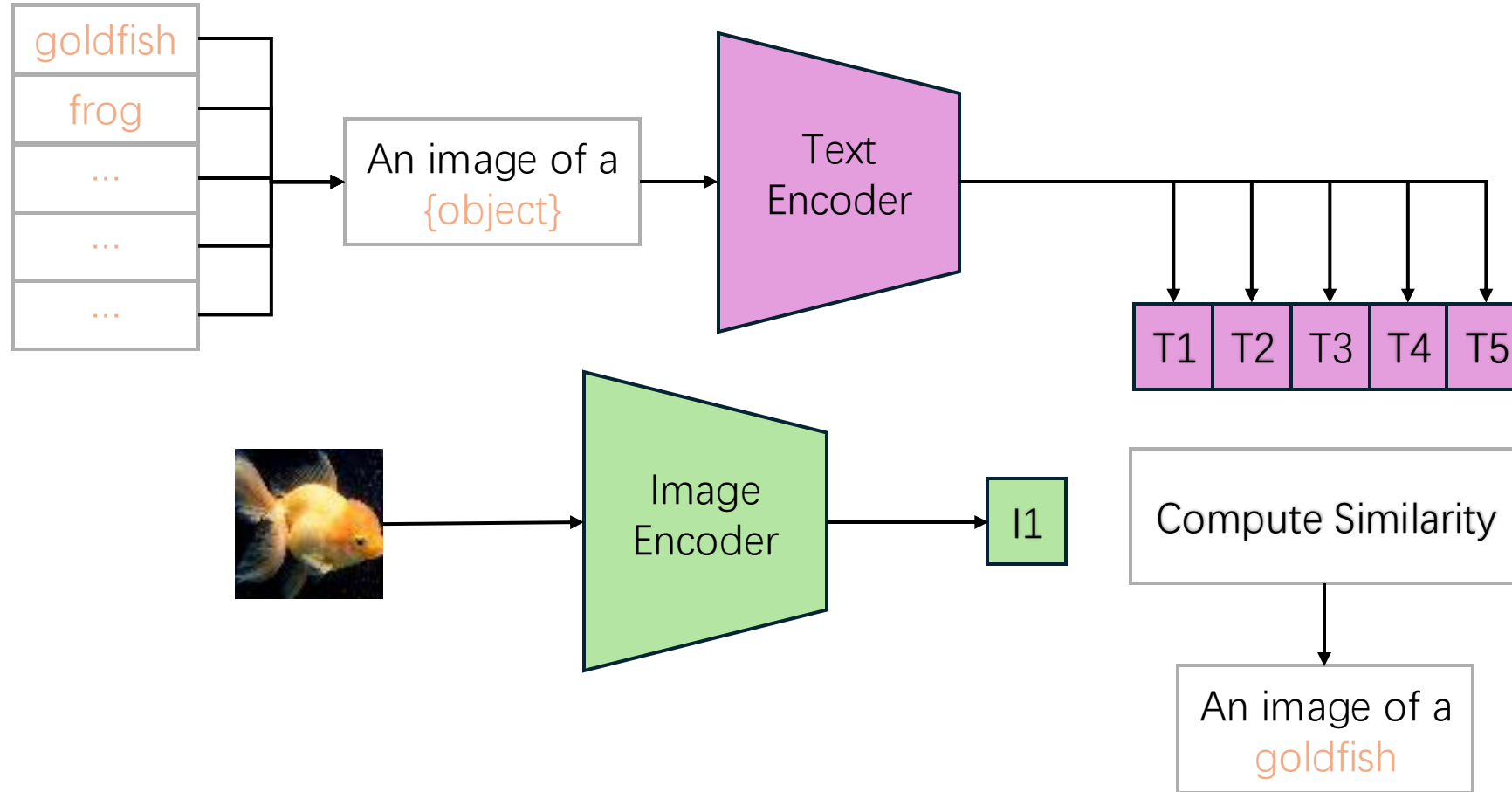
Current Experiment

- Experiments on faster recognition:
on object recognition tasks with a pre-trained vision encoder (e.g. CLIP) to research whether additional familiarity training w/wo LoRA can make representations of the early layers more discriminative of the trained images
(Research has shown that neurons with localized receptive fields in the early visual cortex become sensitive to the global context of familiar images)

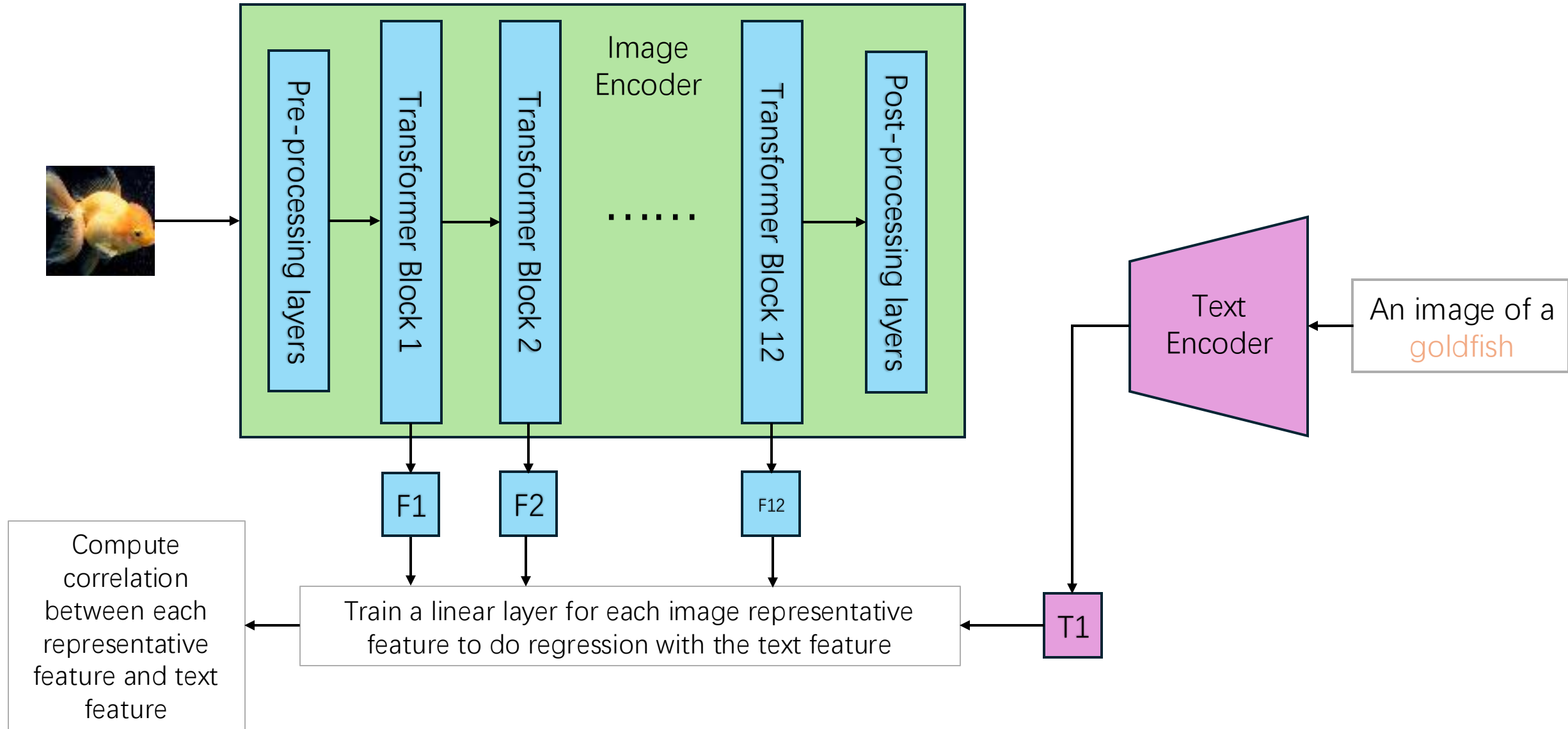
Reference:

Huang, Ge, Suchitra Ramachandran, Tai Sing Lee, and Carl R. Olson. "Neural correlate of visual familiarity in macaque area V2" *Journal of Neuroscience* 38, no. 42 (2018): 8967-8975.

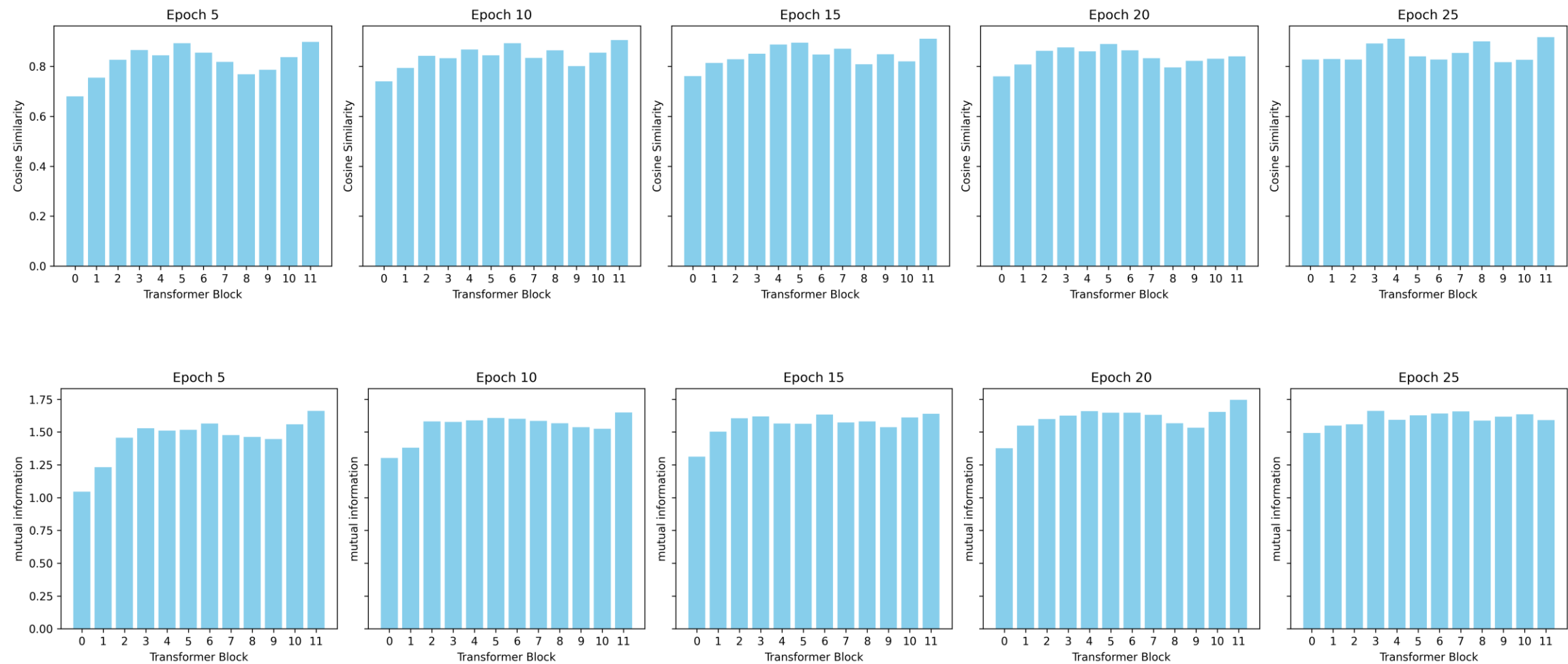
Training Process



Validation Process



Regression Performance of Familiarity Training without LoRA



Regression Performance of Familiarity Training without LoRA

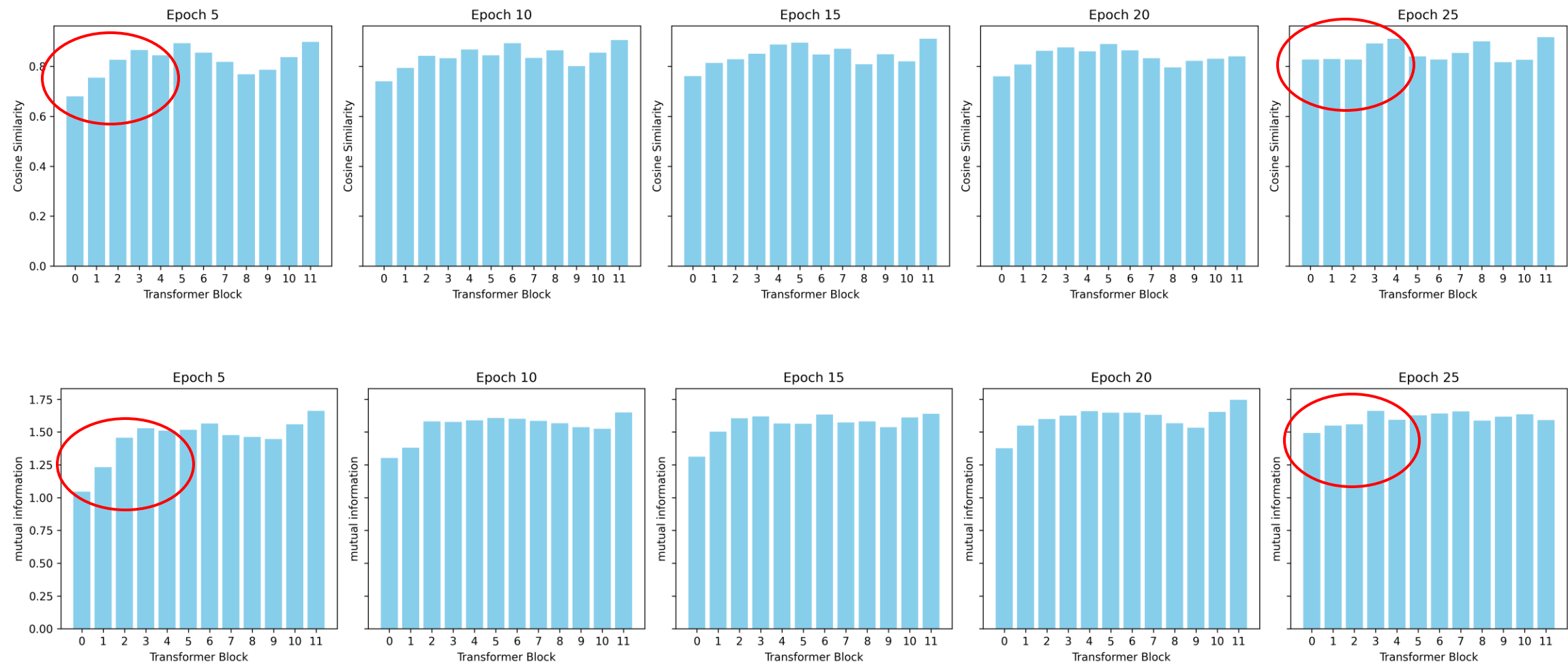
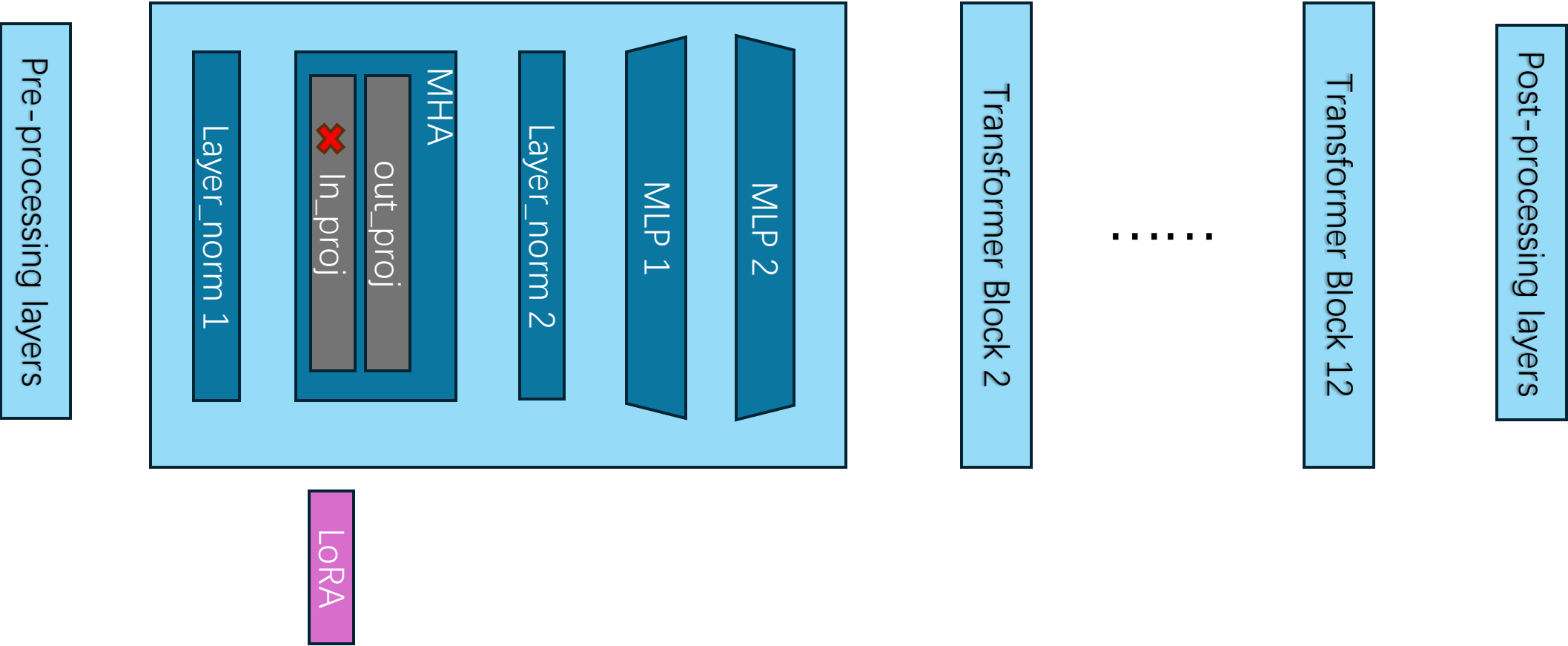


Image Encoder Structure



Performance Comparison

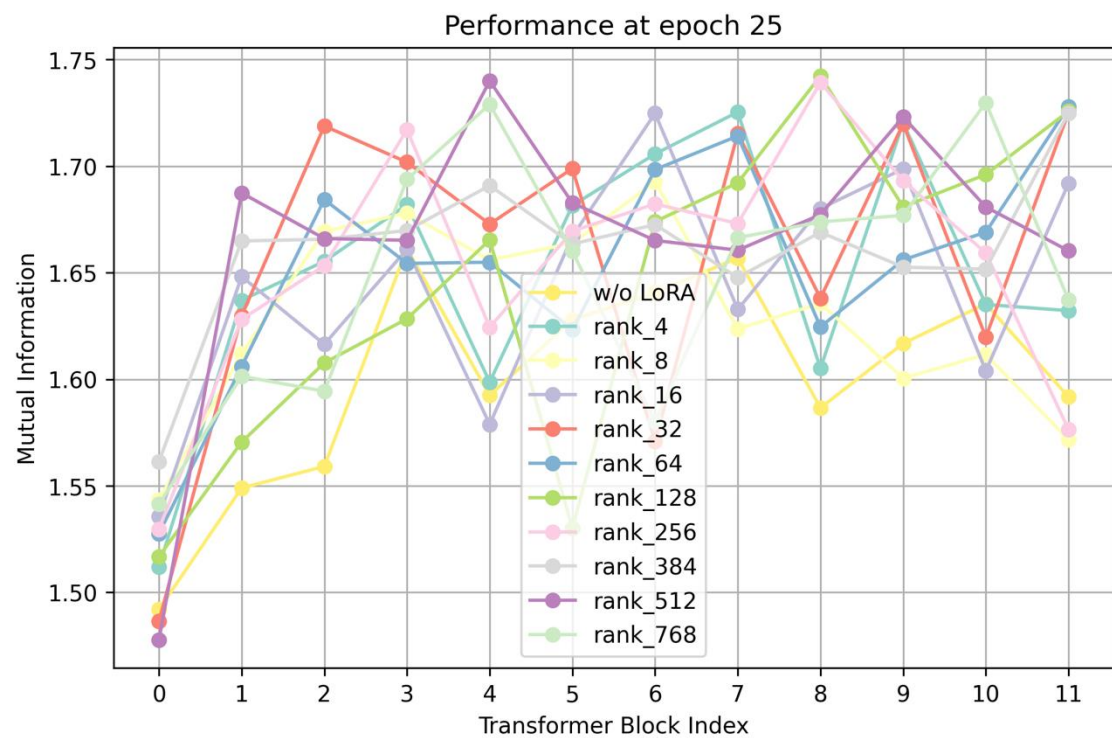
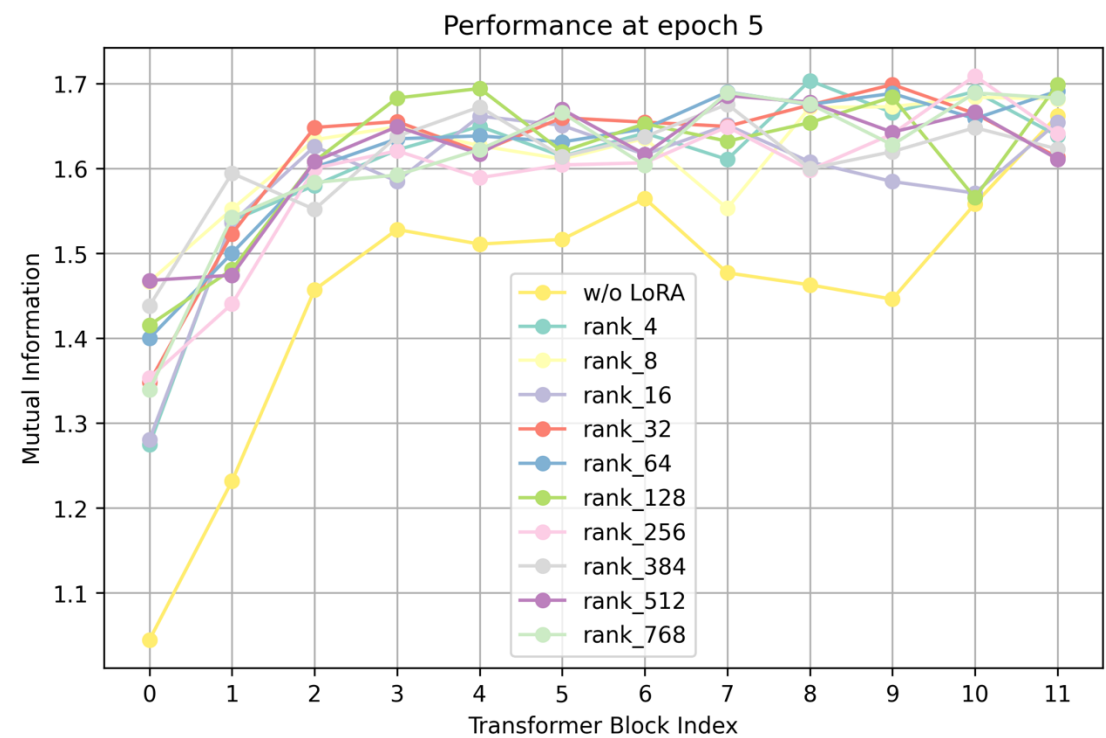
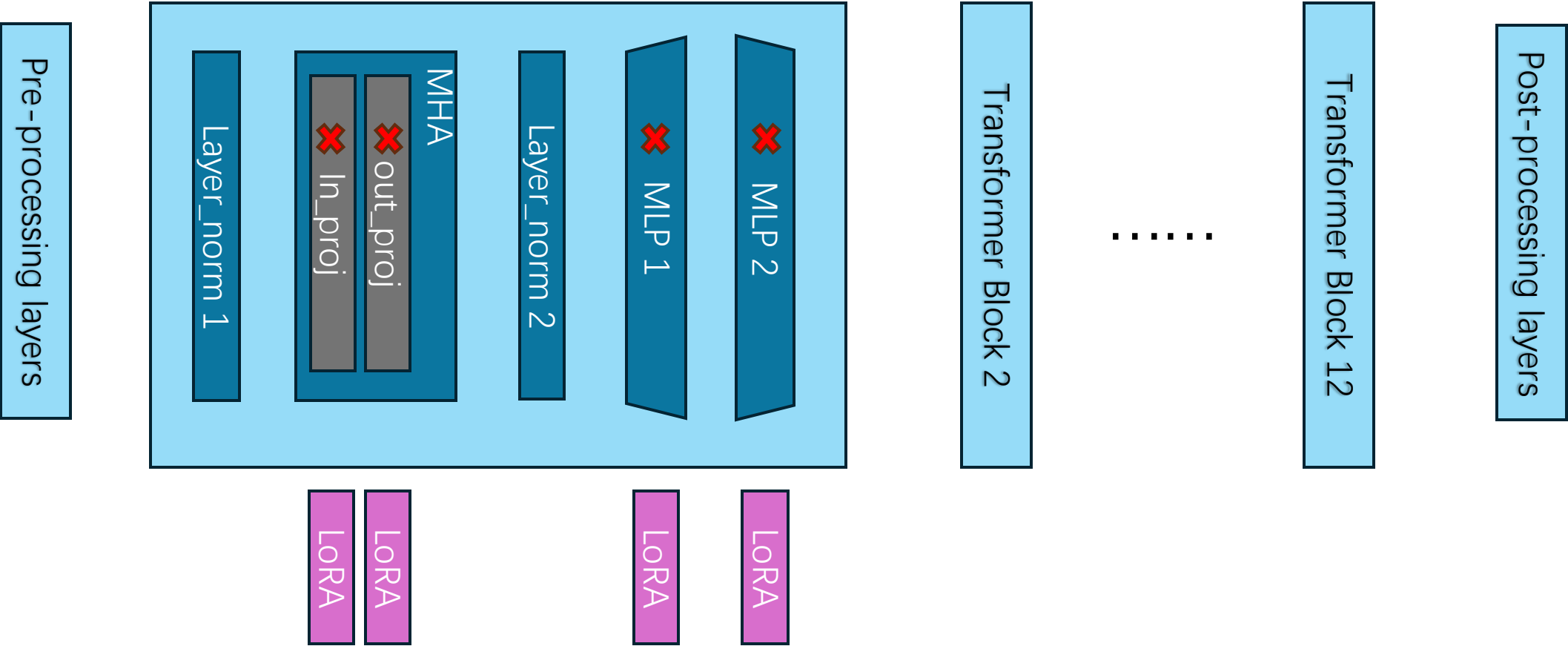


Image Encoder Structure



Performance Comparison

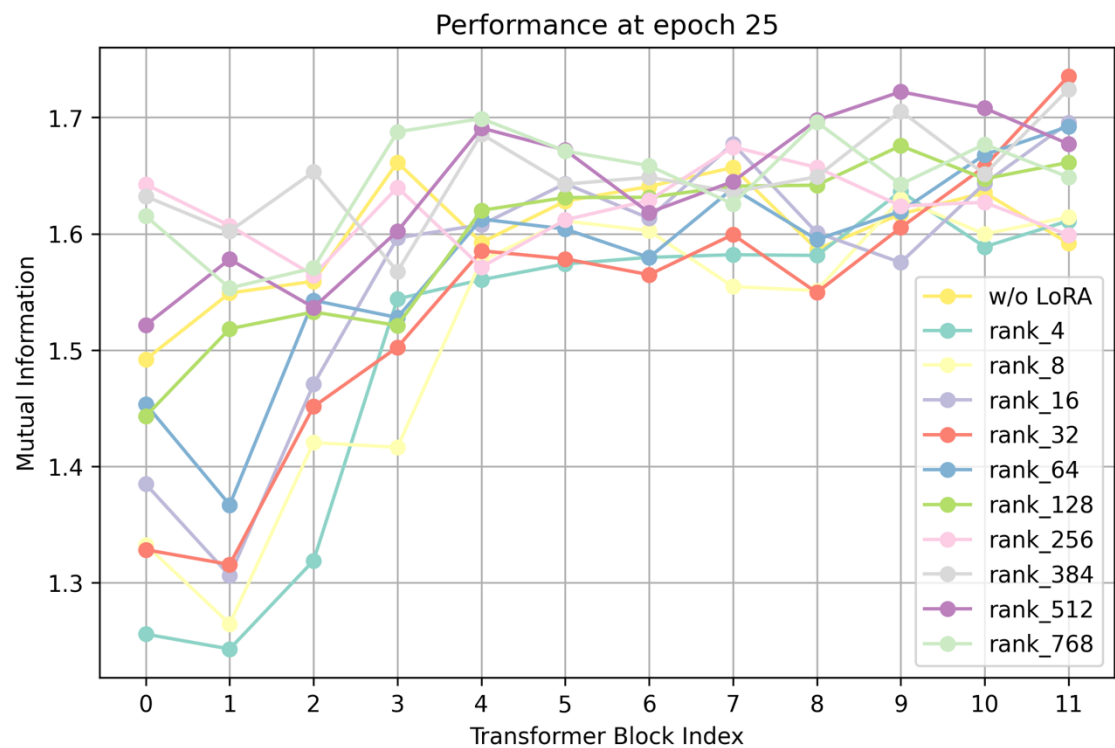
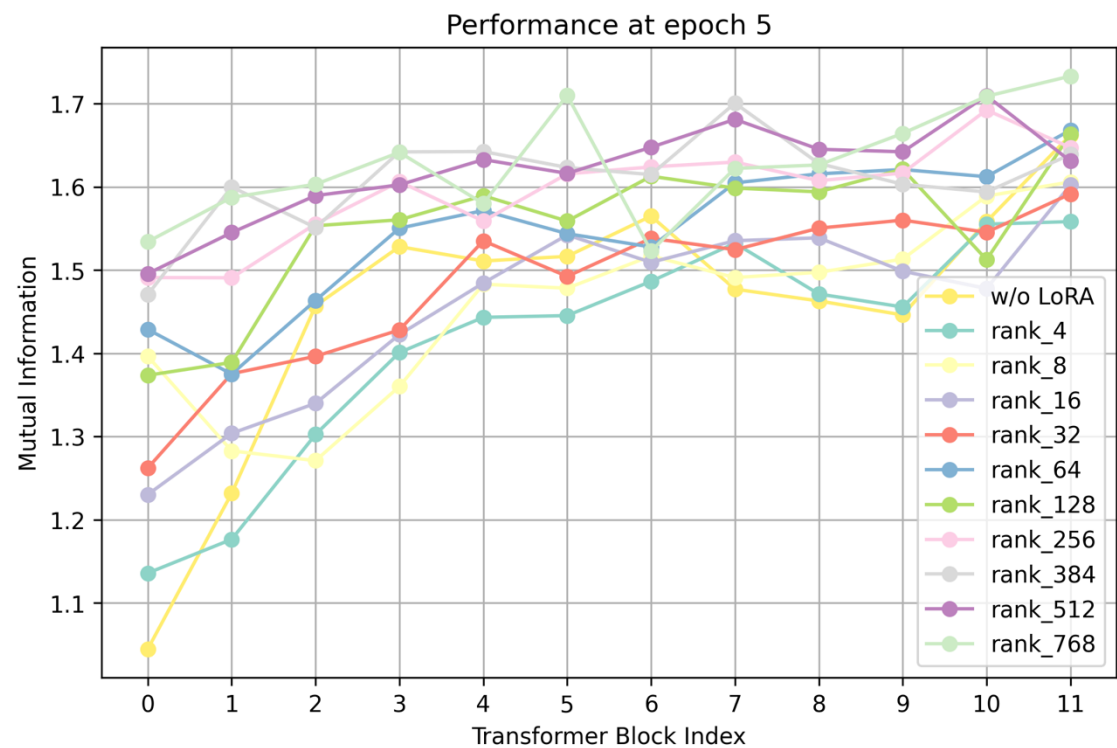
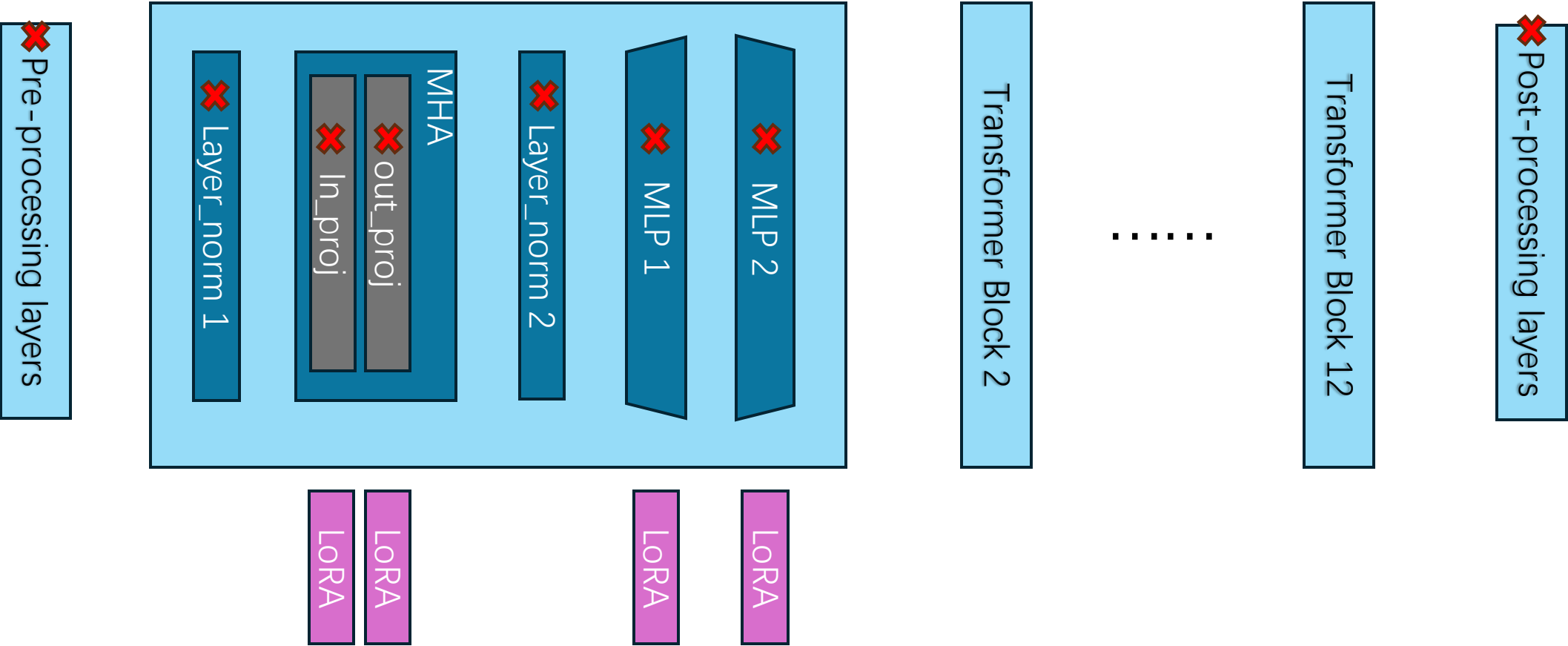
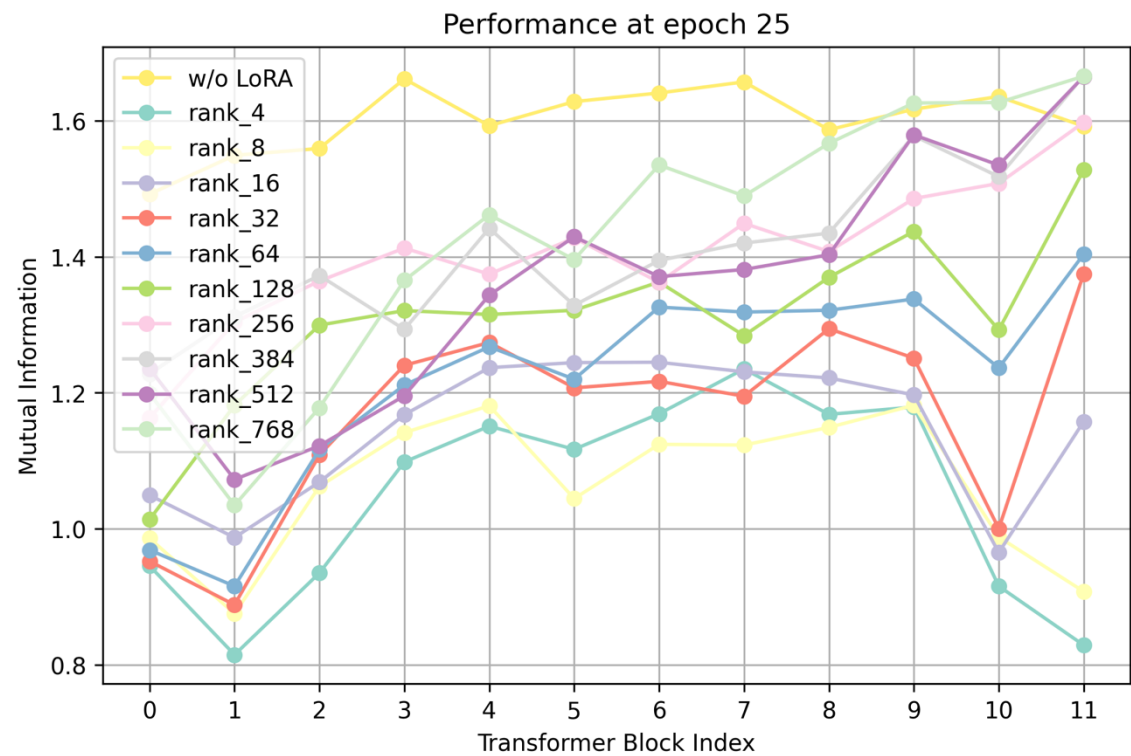
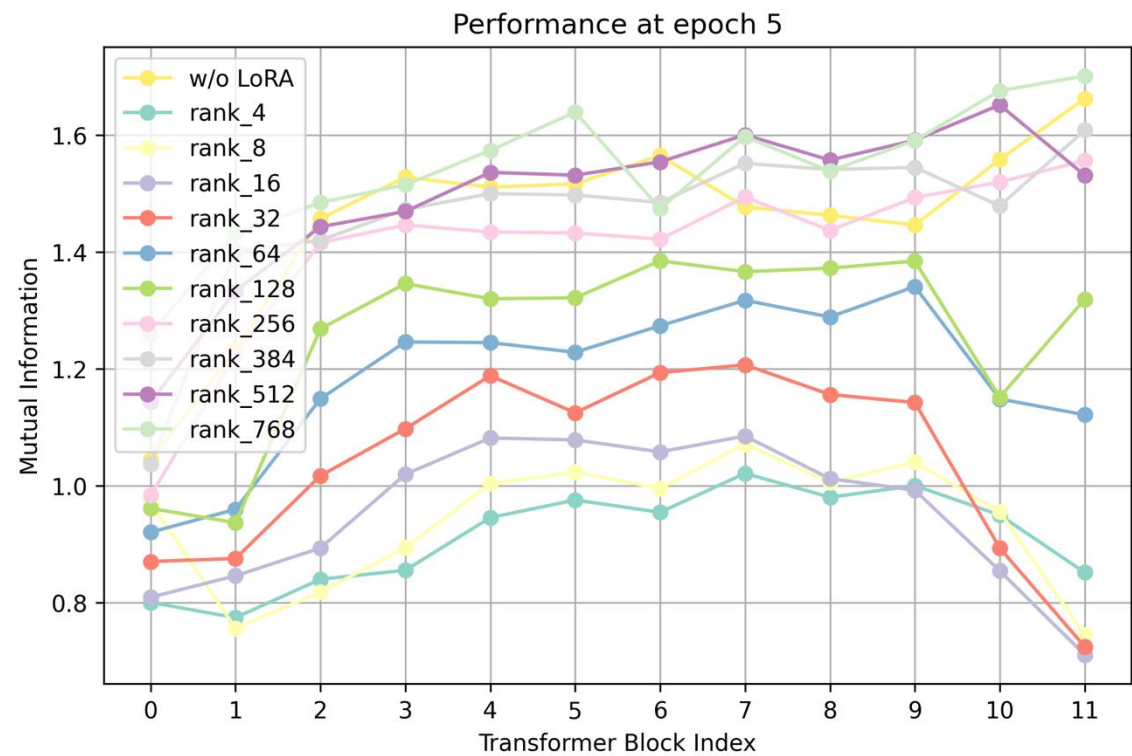


Image Encoder Structure



Performance Comparison



LoRA

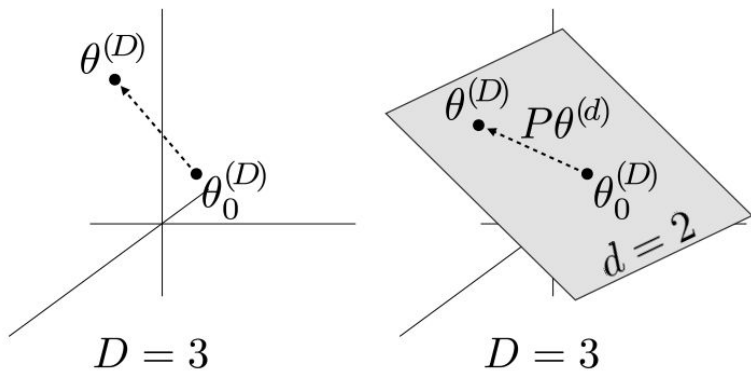
Motivation

- Modern neural networks have a vast number of parameters, often exceeding millions.
- How many parameters are truly needed for optimization and generalization?
- Intuition: Neural networks operate in a smaller, intrinsic subspace.

Intrinsic Dimension:

- Defined as the minimal subspace dimensionality
- Networks are optimized in randomly oriented subspaces instead of the full parameter space

Random Subspace Training



$$\theta_D = \theta_0 + P\theta_d,$$

P is a random projection matrix.

Motivation

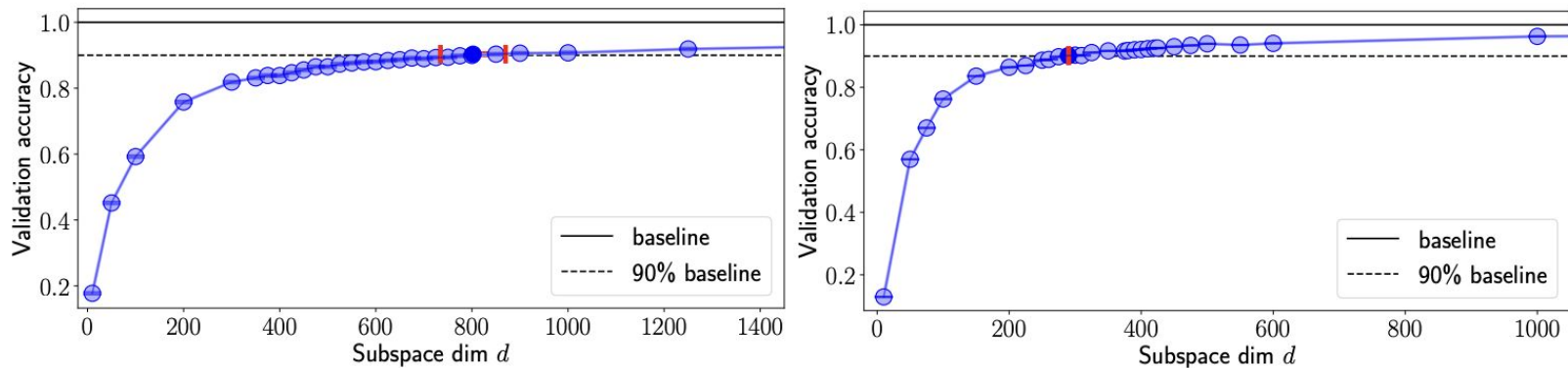


Figure 2: Performance (validation accuracy) vs. subspace dimension d for two networks trained on MNIST: **(left)** a 784–200–200–10 fully-connected (FC) network ($D = 199,210$) and **(right)** a convolutional network, LeNet ($D = 44,426$). The solid line shows performance of a well-trained direct (FC or conv) model, and the dashed line shows the 90% threshold we use to define $d_{\text{int}90}$. The standard deviation of validation accuracy and measured $d_{\text{int}90}$ are visualized as the blue vertical and red horizontal error bars. We oversample the region around the threshold to estimate the dimension of crossing more exactly. We use one-run measurements for $d_{\text{int}90}$ of 750 and 290, respectively.

Intrinsic Dimension across multiple models

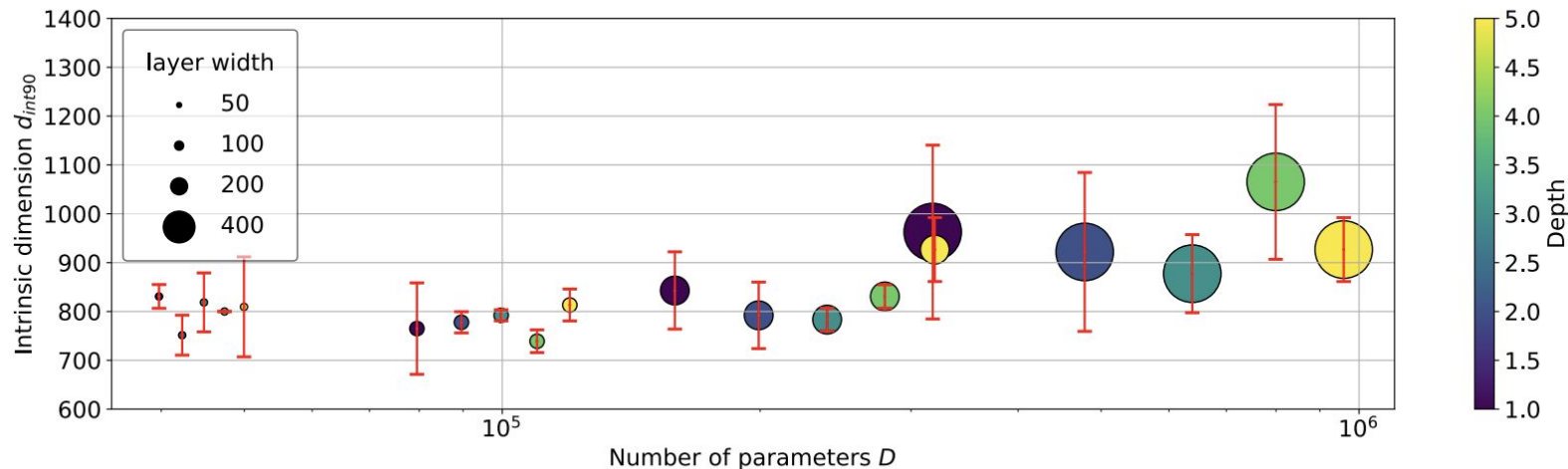


Figure 3: Measured intrinsic dimension $d_{\text{int}90}$ vs number of parameters D for 20 FC models of varying width (from 50 to 400) and depth (number of hidden layers from 1 to 5) trained on MNIST. The red interval is the standard deviation of the measurement of $d_{\text{int}90}$. Though the number of native parameters D varies by a factor of 24.1, $d_{\text{int}90}$ varies by only 1.33, with much of that factor possibly due to noise, showing that $d_{\text{int}90}$ is a fairly robust measure across a model family and that each extra parameter ends up adding an extra dimension directly to the redundancy of the solution. Standard deviation was estimated via bootstrap; see Sec. S5.1.

Random subspace training is quite effective

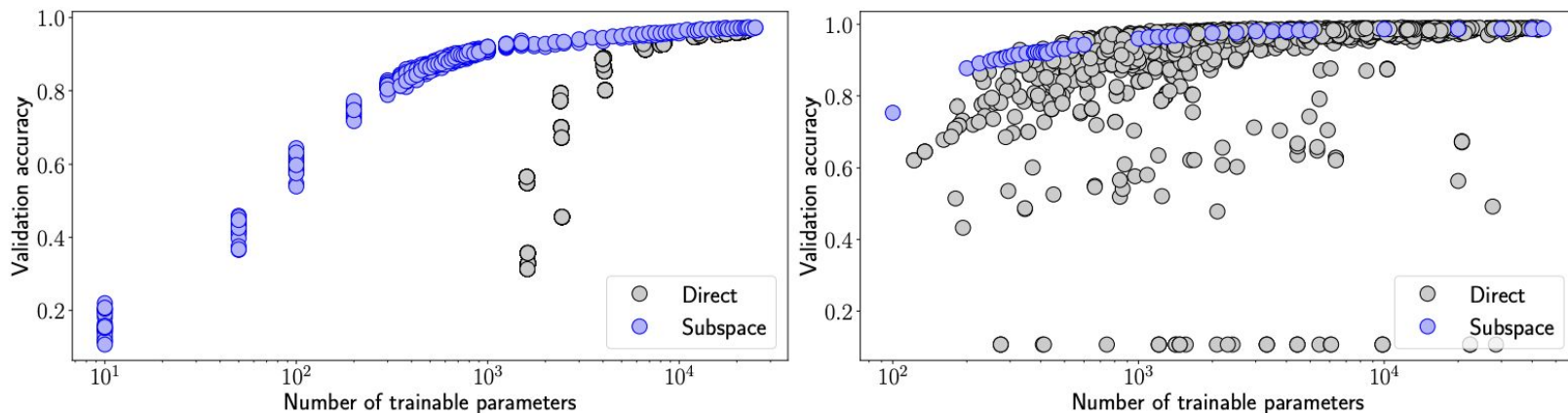
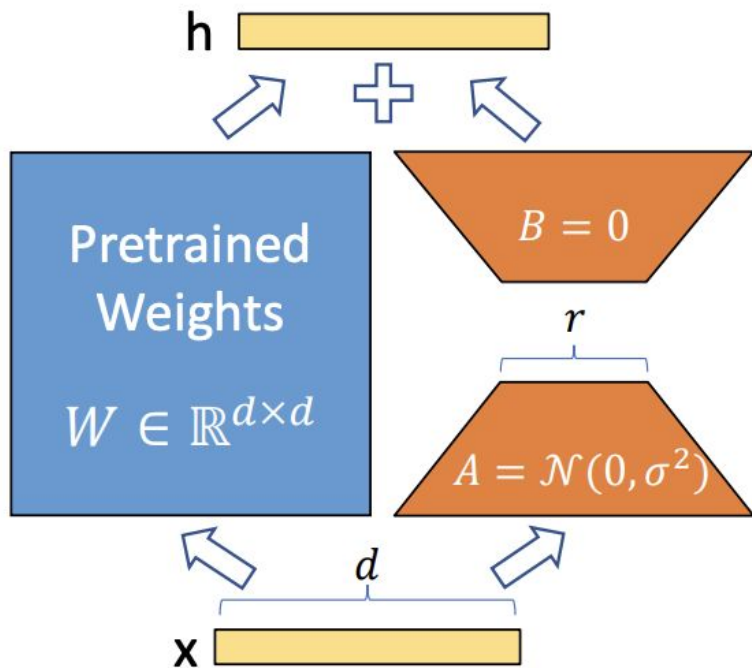


Figure 4: Performance *vs.* number of trainable parameters for **(left)** FC networks and **(right)** convolutional networks trained on MNIST. Randomly generated direct networks are shown (gray circles) alongside all random subspace training results (blue circles) from the sweep shown in Fig. S6. FC networks show a persistent gap in dimension, suggesting general parameter inefficiency of FC models. The parameter efficiency of convolutional networks varies, as the gray points can be significantly to the right of or close to the blue manifold.

LoRA fine tuning



- Weight is very redundant
- Update in a low dimensional subspace
- Learn a A and B via BackProp

$$W_{\text{new}} = W + \Delta W = W + B \cdot A$$

Parameter Efficient Fine-tuning

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB _{base} (FT)*	125.0M	87.6	94.8	90.2	63.6	92.8	91.9	78.7	91.2	86.4
RoB _{base} (BitFit)*	0.1M	84.7	93.7	92.7	62.0	91.8	84.0	81.5	90.8	85.2
RoB _{base} (Adpt ^D)*	0.3M	87.1 $\pm$ 0.0	94.2 $\pm$ 0.1	88.5 $\pm$ 1.1	60.8 $\pm$ 0.4	93.1 $\pm$ 0.1	90.2 $\pm$ 0.0	71.5 $\pm$ 2.7	89.7 $\pm$ 0.3	84.4
RoB _{base} (Adpt ^D)*	0.9M	87.3 $\pm$ 0.1	94.7 $\pm$ 0.3	88.4 $\pm$ 0.1	62.6 $\pm$ 0.9	93.0 $\pm$ 0.2	90.6 $\pm$ 0.0	75.9 $\pm$ 2.2	90.3 $\pm$ 0.1	85.4
RoB _{base} (LoRA)	0.3M	87.5 $\pm$ 0.3	95.1 $\pm$ 0.2	89.7 $\pm$ 0.7	63.4 $\pm$ 1.2	93.3 $\pm$ 0.3	90.8 $\pm$ 0.1	86.6 $\pm$ 0.7	91.5 $\pm$ 0.2	87.2
RoB _{large} (FT)*	355.0M	90.2	96.4	90.9	68.0	94.7	92.2	86.6	92.4	88.9
RoB _{large} (LoRA)	0.8M	90.6 $\pm$ 0.2	96.2 $\pm$ 0.5	90.9 $\pm$ 1.2	68.2 $\pm$ 1.9	94.9 $\pm$ 0.3	91.6 $\pm$ 0.1	87.4 $\pm$ 2.5	92.6 $\pm$ 0.2	89.0
RoB _{large} (Adpt ^P)†	3.0M	90.2 $\pm$ 0.3	96.1 $\pm$ 0.3	90.2 $\pm$ 0.7	68.3 $\pm$ 1.0	94.8 $\pm$ 0.2	91.9 $\pm$ 0.1	83.8 $\pm$ 2.9	92.1 $\pm$ 0.7	88.4
RoB _{large} (Adpt ^P)†	0.8M	90.5 $\pm$ 0.3	96.6 $\pm$ 0.2	89.7 $\pm$ 1.2	67.8 $\pm$ 2.5	94.8 $\pm$ 0.3	91.7 $\pm$ 0.2	80.1 $\pm$ 2.9	91.9 $\pm$ 0.4	87.9
RoB _{large} (Adpt ^H)†	6.0M	89.9 $\pm$ 0.5	96.2 $\pm$ 0.3	88.7 $\pm$ 2.9	66.5 $\pm$ 4.4	94.7 $\pm$ 0.2	92.1 $\pm$ 0.1	83.4 $\pm$ 1.1	91.0 $\pm$ 1.7	87.8
RoB _{large} (Adpt ^H)†	0.8M	90.3 $\pm$ 0.3	96.3 $\pm$ 0.5	87.7 $\pm$ 1.7	66.3 $\pm$ 2.0	94.7 $\pm$ 0.2	91.5 $\pm$ 0.1	72.9 $\pm$ 2.9	91.5 $\pm$ 0.5	86.4
RoB _{large} (LoRA)†	0.8M	90.6 $\pm$ 0.2	96.2 $\pm$ 0.5	90.2 $\pm$ 1.0	68.2 $\pm$ 1.9	94.8 $\pm$ 0.3	91.6 $\pm$ 0.2	85.2 $\pm$ 1.1	92.3 $\pm$ 0.5	88.6
DeB _{XXL} (FT)*	1500.0M	91.8	97.2	92.0	72.0	96.0	92.7	93.9	92.9	91.1
DeB _{XXL} (LoRA)	4.7M	91.9 $\pm$ 0.2	96.9 $\pm$ 0.2	92.6 $\pm$ 0.6	72.4 $\pm$ 1.1	96.0 $\pm$ 0.1	92.9 $\pm$ 0.1	94.9 $\pm$ 0.4	93.0 $\pm$ 0.2	91.3

LoRA is very simple to implement

Pytorch Code Example

```
# These are the weights from the original pretrained model  
self.linear = nn.Linear(in_dim, out_dim, bias=False)
```

```
# These are the new LoRA params. In general rank << in_dim, out_dim  
self.lora_a = nn.Linear(in_dim, rank, bias=False)  
self.lora_b = nn.Linear(rank, out_dim, bias=False)
```

```
# Rank and alpha are commonly-tuned hyperparameters  
self.rank = rank  
self.alpha = alpha
```

```
# Most implementations also include some dropout  
self.dropout = nn.Dropout(p=dropout)
```

```
# The original params are frozen, and only LoRA params are trainable  
self.linear.weight.requires_grad = False  
self.lora_a.weight.requires_grad = True  
self.lora_b.weight.requires_grad = True
```

```
def forward(self, x: torch.Tensor) -> torch.Tensor:  
    # This would be the output of the original model  
    frozen_out = self.linear(x)
```

```
    # lora_a projects inputs down to the much smaller self.rank,  
    # then lora_b projects back up to the output dimension  
    lora_out = self.lora_b(self.lora_a(self.dropout(x)))
```

```
    # Finally, scale by the alpha parameter (normalized by rank)  
    # and add to the original model's outputs  
    return frozen_out + (self.alpha / self.rank) * lora_out
```

Coupling between Magnitude and Direction in LoRA

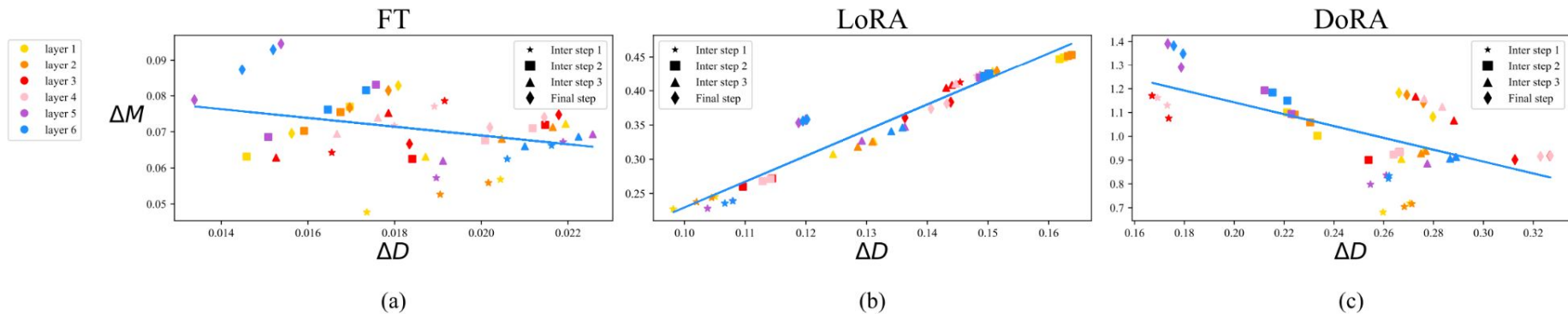


Figure 2. Magnitude and direction updates of (a) FT, (b) LoRA, and (c) DoRA of the query matrices across different layers and intermediate steps. Different markers represent matrices of different training steps and different colors represent the matrices of each layer.

Introducing DoRA, a High-Performing Alternative to LoRA for Fine-Tuning

Jun 28, 2024

By Min-Hung Chen

ICML2024 (Oral)

👍 +29 Like 💬 Discuss (0)

